Algorithms 1

Mingxun Zhou (mingxunz@ust.hk)

COMP2711 Discrete Mathematical Tools

Learning objectives

After this lecture, you should be able to:

1. **Problems and algorithms.** Specify a problem, identify an instance, and read pseudocode.
2. **Running time.** Count basic operations and analyze the worst case.
3. **Asymptotic analysis.** Use O, Ω, and Θ; bound running times using sums.

1 Problems and algorithms

Problem: Specify the inputs and required output.

Instance: One particular input.

Example: GCD. Input: nonnegative integers a, b, not both zero. Output: $\gcd(a, b)$. For instance $(43, 17)$, the answer is 1.

MUST UNDERSTAND Algorithm and correctness

An **algorithm** is a precise sequence of steps for solving a problem. It must stop and return a correct answer on every valid input. Several algorithms may solve the same problem [1, Sections 3.1–3.3].

Pseudocode expresses the steps without choosing a programming language. For example:

EuclideanGCD(a, b)
if $a < b$: swap(a, b)
while $b \neq 0$:
  $(a, b) \leftarrow (b, a \bmod b)$
return a

The arrow $\leftarrow$ means assignment; the pair update uses the old values of both variables. For example,

$$(43, 17) \to (17, 9) \to (9, 8) \to (8, 1) \to (1, 0).$$

Why it works. Each step preserves the GCD and decreases the positive second entry. At $b = 0$, return a. Actual code follows a language's syntax, e.g. Python: `a, b = b, a % b`.

2 Running time and worst-case analysis

MUST UNDERSTAND Running time and the cost model

Running time counts basic operations. In the unit-cost **RAM model**, each assignment, comparison, machine-word arithmetic operation, or memory access costs one unit. Arbitrarily long integers may cost more.

We may instead count a selected operation, such as a comparison. Always specify the input size and the operation being counted.

MUST UNDERSTAND Worst-case running time

For inputs of size n, the worst-case running time is

$$T(n) = \max_{\text{valid instances } I \text{ of size } n} \text{time}(I).$$

This bounds every instance of size n. Best case uses the minimum; average case depends on an input distribution.

2.1 The Alice–Bob guessing game

Alice chooses a secret $a \in \{1, \ldots, n\}$. Bob guesses, and Alice replies "too small", "too big", or "correct". Count one call to Alice as one operation, including the final correct guess.

Linear search guesses $1, 2, 3, \ldots$ until correct. It uses exactly a calls on secret a, so its worst case is $a = n$, requiring n calls.

Binary search guesses the midpoint $g = \lfloor (L + R)/2 \rfloor$ of the remaining interval $[L, R]$:

$$\text{too small: } L \leftarrow g + 1; \qquad \text{too big: } R \leftarrow g - 1.$$

Each failed guess removes at least half the candidates while keeping the secret.

Range	Secret	Guesses made by binary search
$1, \ldots, 16$	13	$8, 12, 14, 13$
$1, \ldots, 16$	1	$8, 4, 2, 1$

Linear search takes thirteen calls in the first example, but only one in the second. Thus comparing a single instance does not establish a worst-case comparison.

Worst-case bound. Before the tth call, the secret remains among at most $n/2^{t-1}$ candidates. Thus

$$1 \le \frac{n}{2^{t-1}} \quad \Longrightarrow \quad t \le \log_2 n + 1.$$

Secret n attains the integer bound. Therefore:

$$T_{\text{linear}}(n) = n, \qquad T_{\text{binary}}(n) = \lfloor \log_2 n \rfloor + 1.$$

2.2 Matrix multiplication

Given two $n \times n$ matrices, compute $C = AB$, where

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

Multiply(A, B)
for $i = 1, \ldots, n$:
  for $j = 1, \ldots, n$:
    $c_{ij} \leftarrow 0$
    for $k = 1, \ldots, n$:
      $c_{ij} \leftarrow c_{ij} + a_{ik} b_{kj}$
return C

Count: n^2 entries $\times$ n multiplications each $= n^3$ multiplications.

There are also n^3 additions and $n^3 + n^2$ assignments to C. Asymptotic analysis describes growth without tracking every such detail.

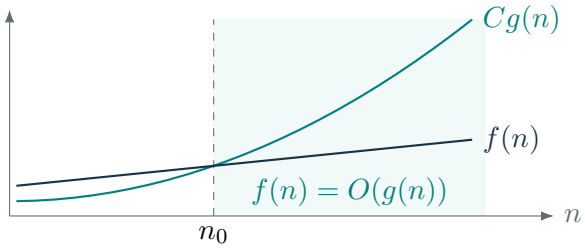
3 Asymptotic bounds

MUST UNDERSTAND Big-O: an upper bound

For eventually nonnegative functions f, g, write $f(n) = O(g(n))$ if there are constants $C > 0$ and n_0 such that

$$f(n) \leq Cg(n) \qquad \text{for every } n \geq n_0.$$

The constants do not depend on n. The inequality may fail before n_0.



For example, $5n^2 + 2n + 1 \leq 8n^2$ for $n \geq 1$, so $C = 8, n_0 = 1$ prove $5n^2 + 2n + 1 = O(n^2)$.

Function	A Big-O bound	Growth type
7	$O(1)$	Constant
$\log_2 n + 1$	$O(\log n)$	Logarithmic
$3n + 10$	$O(n)$	Linear
$n \log_2 n + 4n$	$O(n \log n)$	Quasilinear
$5n^2 + 2n + 1$	$O(n^2)$	Quadratic
$2n^3 + 5n^2 + 1$	$O(n^3)$	Cubic

A function has many valid upper bounds: n^2 is also $O(n^3)$ and $O(n^4)$. Changing a logarithm's fixed base changes only a constant factor.

Some less obvious examples are:

Function	Bound	Reason
$n!$	$O(n^n)$	Each factor is at most n.
$n! + 2^n$	$O(n!)$	$2^n \le n!$ for $n \ge 4$.
$n^2(2 + (-1)^n)$	$O(n^2)$	The multiplier is 1 or 3.
$\sqrt{n^2 + n}$	$O(n)$	At most $\sqrt{2}\,n$ for $n \ge 1$.
2^{n+10}	$O(2^n)$	Equal to 1024×2^n.
2^{2n}	$O(4^n)$	Its ratio to 2^n tends to infinity.

In the last row, 2^{2n} is not $O(2^n)$: no fixed constant can bound the ratio 2^n for all large n.

For matrix multiplication, each of the n^3 inner iterations does constant work. Initialization and loop overhead also fit within a constant multiple of n^3, giving $O(n^3)$ time in the stated arithmetic model.

3.1 Finding a duplicate

Given $A[1..n]$, return a repeated value or report no match. For $[1, 2, 5, 6, 2, 10]$, the repeated value is 2.

Try every pair. For each i, compare $A[i]$ with $A[j]$ for $j > i$, stopping at a match. Every pair occurs once, so a repeated value is found if one exists. When all values are distinct, every comparison is needed:

$$W(n) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} = O(n^2).$$

Remember values already seen. If every value lies in $\{1, \ldots, n\}$, initialize *SeenBefore*$[1..n]$ to No. For each input value x, return x if *SeenBefore*$[x]$ is Yes; otherwise set it to Yes. Before each step, the array records exactly the values seen earlier, proving correctness. Initialization and scanning each take $O(n)$ time; extra space is $O(n)$.

What if the values are unbounded?

- A table for values $1, \ldots, U$ needs $O(U)$ space and initialization.
- Sorting and checking adjacent values takes $O(n \log n)$ comparisons.
- Hashing can give expected $O(n)$ time, assuming constant-time hashing/equality and suitable randomized hashing.

3.2 Lower bounds, tight bounds, and strict bounds

MUST UNDERSTAND Big-Omega and Big-Theta

For eventually nonnegative f, g:

$$f(n) = \Omega(g(n)) \iff cg(n) \le f(n) \text{ for all } n \ge n_0,$$
$$f(n) = \Theta(g(n)) \iff c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0,$$

for some positive constants c, c_1, c_2 and a threshold n_0. A tight bound requires both an upper and a lower bound of the same order.

For example, for $n \ge 1$,

$$3n^2 \le 3n^2 + 5n + 7 \le 15n^2,$$

so $3n^2 + 5n + 7 = \Theta(n^2)$. The symbols describe bounds on a chosen function: they do not themselves specify best, worst, or average case. Linear search has best-case cost $\Theta(1)$ and worst-case cost $\Theta(n)$.

For $f(n) \ge 0$ and $g(n) > 0$, the strict bounds are

$$f(n) = o(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0, \qquad f(n) = \omega(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty.$$

Thus $100n^2 = o(n^3)$ and $n^3 = \omega(100n^2)$. In contrast, $3n^2$ is $O(n^2)$ but not $o(n^2)$.

4 Proving bounds for sums

4.1 Geometric series

For $k \ge 0$ an integer and $r \ne 1$,

$$S = 1 + r + \cdots + r^k = \frac{r^{k+1} - 1}{r - 1}.$$

Proof. Multiply by r and subtract:

$$rS - S = (r + r^2 + \cdots + r^{k+1}) - (1 + r + \cdots + r^k) = r^{k+1} - 1.$$

For $r = 1$, the sum is $k + 1$.

Example. An algorithm does $1, 2, 4, \ldots, n$ operations in successive stages, where $n = 2^k$.

$$T(n) = 1 + 2 + 4 + \cdots + n = 2n - 1 = \Theta(n).$$

All stages together cost less than twice the final stage.

4.2 The harmonic sum

Theorem 4.1. *For $H_n = \sum_{i=1}^{n} 1/i$, we have $H_n = \Theta(\log n)$.*

For $n = 2^k$, group the terms as

$$H_{2^k} = 1 + \underbrace{\frac{1}{2}}_{1\text{ term}} + \underbrace{\frac{1}{3} + \frac{1}{4}}_{2\text{ terms}} + \underbrace{\frac{1}{5} + \cdots + \frac{1}{8}}_{4\text{ terms}} + \cdots .$$

Block j has 2^{j-1} terms with denominators $2^{j-1}+1, \ldots, 2^j$. Its sum B_j satisfies

$$\frac{1}{2} = 2^{j-1}\frac{1}{2^j} \le B_j \le 2^{j-1}\frac{1}{2^{j-1}} = 1.$$

There are k blocks, so $1 + k/2 \le H_{2^k} \le 1 + k$. For general $n \ge 2$, set $k = \lfloor \log_2 n \rfloor$. Then

$$1 + \frac{k}{2} \le H_{2^k} \le H_n \le H_{2^{k+1}} \le k + 2.$$

Using $\log_2 n - 1 \le k \le \log_2 n$ gives

$$\tfrac{1}{2}\log_2 n \le H_n \le 3\log_2 n \qquad (n \ge 2),$$

which proves both bounds.

4.3 A sum of logarithms

Let $S_n = \sum_{i=1}^{n} \log_2 i = \log_2(n!)$. Each term is at most $\log_2 n$, so $S_n \le n \log_2 n$. For the lower bound, keep the latter half of the terms. There are at least $n/2$ terms with $i \ge n/2$, so for $n \ge 4$,

$$S_n \ge \frac{n}{2}\log_2(n/2) \ge \frac{n}{4}\log_2 n.$$

Therefore $S_n = \Theta(n \log n)$.

The same argument analyzes a loop that, for each $i = 1, \ldots, n$, doubles j from 1 until $j > i$. Its total iterations are

$$\sum_{i=1}^{n} (\lfloor \log_2 i \rfloor + 1) = \Theta(n \log n).$$

The arithmetic-series formula is

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

Sequential stages add their costs; repeated work sums the per-iteration costs. If the same work is repeated n times, multiply its cost by n.

5 Optional reading

5.1 Fast exponentiation

Task: Compute $a^b \bmod m$, where $b \geq 1$ and $m \geq 2$.

1. **Extract the bits.** Write $b = \sum_{i=0}^{L-1} b_i 2^i$, where $L = \lfloor \log_2 b \rfloor + 1$. Repeatedly record $b_i = b \bmod 2$ and update $b \leftarrow b \div 2$.
2. **Accumulate.** Start with $r = 1$, $s = a \bmod m$. For each bit b_i, multiply r by s modulo m if $b_i = 1$, then square s modulo m.

Correctness. At step i, $s \equiv a^{2^i} \pmod{m}$. Hence

$$r \equiv \prod_{i:b_i=1} a^{2^i} = a^b \pmod{m}.$$

Example. $13 = 8 + 4 + 1$. The residues of $2^1, 2^2, 2^4, 2^8$ modulo 5 are $2, 4, 1, 1$, so $2^{13} \equiv 1 \times 1 \times 2 \equiv 2$.

Cost. Two passes of L constant-cost steps: $O(\log b)$ arithmetic operations. For $b = 0$, return 1.

5.2 Input length and faster matrix multiplication

An integer $N \geq 1$ uses $\lfloor \log_2 N \rfloor + 1$ bits. A loop with N iterations can therefore be exponential in the input's bit length. Polynomial time means $O(L^c)$ for a fixed constant c, where L is the encoded input length.

The cubic matrix algorithm is not a lower bound for the problem. The deck's October 2026 research extension discusses OpenAI's nonconstructive exponent bound 9/4 [2] and Pass's explicit construction with $O(n^{2.25+\varepsilon})$ arithmetic operations for every fixed $\varepsilon > 0$ [3]. These are arithmetic-operation bounds; they do not by themselves establish practical performance.

5.3 Further algorithm examples

Array search. Linear search checks entries in order: worst-case $\Theta(n)$. Binary search halves a sorted array's remaining interval: worst-case $\Theta(\log n)$.

Sorting. Insertion sort inserts each next value into a sorted prefix: best-case $\Theta(n)$, worst-case $\Theta(n^2)$. Fixed-pass bubble sort swaps inverted neighbors and uses $n(n-1)/2$ comparisons. Algorithms 2 gives the full analyses.

String matching. Compare a length-m pattern at all $n - m + 1$ possible starts in a length-n text. This checks every occurrence using at most $(n - m + 1)m$ comparisons. Example: `ABA` occurs at positions $1, 3, 5$ in `ABABABA`.

Greedy scheduling. To select the most nonoverlapping intervals:

1. Take the compatible interval that finishes earliest.
2. Repeat on the intervals starting after it finishes.

Why it works: Replace an optimal schedule's first interval by this choice. It finishes no later, so all later intervals still fit. Repeat the exchange.

For $[0,4), [1,3), [3,5), [4,7), [5,6), [6,8)$, select $[1,3), [3,5), [5,6), [6,8)$.

Greedy rules need their own proofs. With coins $1, 3, 4$, largest-first makes 6 as $4+1+1$; $3+3$ is better.

Design patterns.

Method	Idea
Brute force	Check all candidates.
Greedy	Commit to a locally preferred choice.
Divide and conquer	Solve smaller instances and combine their answers.
Dynamic programming	Reuse answers to overlapping subproblems.

5.4 Finding, verifying, and computability

Algorithms 3 develops these concepts:

P: Decision problems solvable in polynomial time.

NP: Decision problems with polynomial-length Yes certificates verifiable in polynomial time.

Reduction: A polynomial-time transformation preserving Yes and No answers.

NP-complete: A problem in NP to which every NP problem reduces.

Example: for subset sum on $3, 5, 9, 12$ with target 17, selecting $5, 12$ certifies Yes. A polynomial-time algorithm for one NP-complete problem would give P = NP.

The halting problem asks whether a program stops on a given input. No algorithm solves every instance.

Proof. Suppose $H(P, x)$ predicts whether P halts on x. Define $D(P)$ to loop forever if $H(P, P)$ predicts halting, and to halt otherwise. Then $D(D)$ contradicts either prediction.

6 Exercises

1. Trace binary guessing for $n = 16$ and secrets 2 and 15.
2. Give constants proving $4n^2 + 3n + 8 = \Theta(n^2)$.
3. Count the body executions when $i = 1, \ldots, n$ and, for each i, $j = 1, \ldots, n - i$.
4. Decide whether $n^2 = O(n)$, $3^n = O(4^n)$, and $4^n = O(3^n)$.
5. Does $f(n) = O(n^2)$ imply $f(n) = \Theta(n^2)$? Give an example.
6. Prove that $\sum_{i=1}^{n} \log_2 i = \Theta(n \log n)$ using only the latter half for the lower bound.
7. What changes in the duplicate-finding problem when the values are not bounded by n?
8. Explain the difference between an exponential-time algorithm and an undecidable problem.

Answers

1. The guesses are $8, 4, 2$ and $8, 12, 14, 15$, respectively.
2. For $n \geq 1$, $4n^2 \leq 4n^2 + 3n + 8 \leq 15n^2$. Use $c_1 = 4, c_2 = 15, n_0 = 1$.
3. $\sum_{i=1}^{n}(n - i) = n(n - 1)/2 = \Theta(n^2)$.
4. False, true, false. The relevant ratios are n, $(3/4)^n$, and $(4/3)^n$.
5. No. For example, $f(n) = n$ is $O(n^2)$ but not $\Omega(n^2)$.
6. For $n \geq 4$, $(n/4)\log_2 n \leq \sum_i \log_2 i \leq n \log_2 n$; see the worked proof above.
7. An array of flags may no longer have size $O(n)$. Sorting gives $O(n \log n)$ comparisons; hashing can give expected linear time under the assumptions stated in the duplicate example.
8. An exponential-time algorithm still terminates and solves its problem. An undecidable problem has no algorithm that terminates with the correct answer on every input. One slow algorithm does not prove that its problem is inherently slow.

References

[1] Kenneth H. Rosen. *Discrete Mathematics and Its Applications.* McGraw-Hill Education, 8th edition, 2019.

[2] OpenAI. An upper bound of 9/4 for the matrix multiplication exponent, October 2026. October 2. Official preprint.

[3] Rafael Pass. Matrix multiplication from polynomial packings: A constructive and elementary proof of $\omega \leq 9/4$. arXiv:2610.28627v2, October 2026. Version 2, October 30.